



Gateway Technical

LLM

This lecture builds a mental model of how modern LLM systems are structured and adapted; implementation details are illustrative, not examinable.





7.0 Module orientation: large language models

Why this module matters

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal orange lines stacked vertically.
- Large language models (LLMs) and foundation models mark a major shift in how we build and deploy AI systems.
 - Instead of training narrow models for specific tasks, we now adapt massive pretrained models for a wide variety of uses.
 - Understanding these models is essential for both technical work and strategic decision-making in organizations.

Guiding questions



Keep these things in mind:

1. What makes foundation models different from traditional task-specific models?
2. How do Transformers enable scale and emergent capabilities?
3. What are the main strategies to adapt LLMs to new tasks?
 - Prompting
 - Fine-tuning
 - Retrieval-augmented generation (RAG)
4. How do we balance performance and cost concerns in practice?
5. Where are the boundaries of current technology, and what challenges remain?



7.1 What are foundation models and LLMs?

Definitions, selecting, adapting, misconceptions.



AI has moved from specialized models toward broadly adaptable foundation models.

- 
- Traditional AI built one model per task (e.g., sentiment classifier, translator, summarizer).
 - Each required **unique data, architecture, and retraining**.
 - **Foundation models** emerged as general-purpose systems: trained once on diverse data, adapted many times.

Foundation models learn generalizable representations from massive, diverse data.

- 
- A **foundation model** is a large-scale pretrained model trained on **massive, diverse, and unlabeled** data.
 - Acts as a **base layer** for downstream applications via prompting, fine-tuning, or retrieval-augmented generation (RAG).
 - Provides **transferable representations** that generalize across domains.

Large language models specialize in language tasks using predictive training objectives

- 
- LLMs are **language-focused foundation models** trained on massive text corpora.
 - Objective: predict the next token (causal) or fill masked tokens (masked).
 - Enable tasks such as summarization, Q&A, translation, and reasoning (often without retraining.)

Modern LLMs unify many tasks that once required separate models.

Traditional NLP (pre-2018)	Foundation / LLM Era
Model per task (e.g., sentiment, NER, translation)	One model, many tasks
Limited labeled data	Massive unlabeled data
Distinct architectures	Shared Transformer backbone
Narrow generalization	Broad cross-task generalization

Different LLM families use varied architectures and objectives to excel at different tasks.

Model Family	Architecture	Objective	Core Contribution	Successors / Variants
GPT	Decoder-only	Causal LM	Open-ended generation and reasoning	InstructGPT → ChatGPT → GPT-4
BERT	Encoder-only	Masked LM	Deep bidirectional representations	RoBERTa → DeBERTa
T5	Encoder-decoder	Text-to-text	Unified text-in/text-out framework	Flan-T5
PaLM	Decoder-only	Causal LM	Large-scale multilingual reasoning	Gemini family
Claude	Decoder-only	Causal LM + alignment	Conversational helpfulness and safety	Constitutional AI lineage
LLaMA	Decoder-only	Open-weight general model	Democratized large-model research	Llama 2 / Llama 3
Mistral	Decoder-only	Efficient dense + Mixture-of-Experts	Compact, fast open models	Mixtral series

LLM capabilities evolve through interconnected research lineages.

- Foundation model development follows **interconnected research families**:
 - **GPT → InstructGPT → ChatGPT → GPT-4** (scaling + instruction-tuning)
 - **BERT → RoBERTa → DeBERTa** (representation depth)
 - **T5 → FLAN-T5** (task-unified, instruction-tuned)
 - **PaLM → Gemini** (multilingual, multimodal expansion)
 - **Claude** (alignment-driven, “Constitutional AI”)
 - **LLaMA → Mistral → Mixtral** (open-weight efficiency movement)

Key takeaways:

- **Shared architecture patterns** accelerate iteration and cross-pollination.
- **Instruction-tuning and alignment** transform base LMs into assistants.
- **Open-weight ecosystems** (LLaMA, Mistral) now drive accessibility and experimentation.

Pretraining teaches models representations through self-supervised objectives

- 
- Uses **self-supervised learning**. Models learn from raw text without labels.
 - Objective functions: next-token prediction, masked-token prediction, contrastive learning.
 - Produces embeddings that encode syntax, semantics, and context.

Self-supervised pretraining unlocked scalable, adaptable language intelligence.

- 
- Pretraining replaces costly labeled data with **self-supervision** on raw text.
 - Massive scale → richer internal representations and emergent abilities.
 - One model now supports many downstream tasks with minimal additional effort.

Different model architectures support understanding, generating, or transforming text.

Category	Example Models	Typical Use
Encoder-only	BERT, RoBERTa	Representation, extraction
Decoder-only	GPT, Claude, PaLM	Generation, reasoning
Encoder-decoder	T5, FLAN-T5	Text-to-text tasks

These categories are more about where a model “sits” in the processing pipeline. It is not a list of which is better

Foundation models differ based on whether they process text, vision, audio, or multimodal input.

Modality	Example Foundation Models	Core Input
Text	GPT, Claude, Gemini-1.5	Tokens
Vision	CLIP, DINOv2	Pixels
Audio/Speech	Whisper, Bark	Waveforms
Multimodal	Gemini, LLaVA, Kosmos-2	Mixed input (text + vision + audio)

Each modality requires a different representation before learning can happen.

Different licensing models balance transparency, control, and flexibility.

Access Type	Characteristics	Example Providers	Implications
Closed API	Proprietary weights; usage via hosted endpoints only	OpenAI, Anthropic	Limited transparency; strong safety layers
Open weights	Downloadable and modifiable weights; community fine-tuning possible	Meta (LLaMA), Mistral	Enables research, customization; requires governance
Fully open	Weights + training data + scripts available	Falcon, Pythia	Maximizes reproducibility; raises IP & security concerns

LLMs operate within a rapidly expanding ecosystem of models and tooling



Selecting an LLM requires balancing capability, cost, and constraints.



Three complementary strategies allow us to adapt foundation models to new tasks.

Large language models can be customized or extended through **three complementary approaches**:

1. **Prompting** – steer the model's behavior through natural language instructions.
 - No retraining, immediate iteration.
2. **Fine-tuning / PEFT** – retrain or lightly adjust parameters for domain- or task-specific use.
 - Higher control, higher cost.
3. **Retrieval-Augmented Generation (RAG)** – connect the model to external, updateable knowledge sources.
 - Keeps responses current and grounded.

Comparing adaptation strategies: cost, control, and scalability

Method	Retraining Required?	Control over Output	Cost / Complexity	Scalability	Best For
Prompting	✗ None	Low–Moderate	💰 Low	High	Rapid experimentation, changing tasks
Fine-tuning / PEFT	✓ Partial or full	High	💰💰–💰💰💰	Moderate	Stable, domain-specific applications
RAG	✗ None (external data link)	Moderate	💰💰	High	Knowledge-grounded or dynamic data systems

Common misconceptions obscure how LLMs truly work and behave.

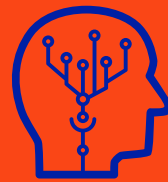




7.2 Transformers: From RNNs to Transformers Why They Matter

Transformers revolutionized sequence modeling by enabling scalable parallelism.

- Transformers replaced RNNs and LSTMs as the dominant architecture for sequence modeling.
- They overcame key limitations: sequential bottlenecks, vanishing gradients, and difficulty modeling long dependencies.
- Parallel attention enabled both **scaling** and **generalization**, the foundation of LLMs.



Transformers are just massive, learned relevance engines operating over tokens.

Earlier models struggled with long-range dependencies and sequential bottlenecks.

- 
- **Sequential processing:** must handle tokens one at a time → slow for long inputs.
 - **Vanishing/exploding gradients:** long dependencies fade or explode during training.
 - **Memory bottlenecks:** fixed-length hidden states can't capture global context.
 - Even LSTMs and GRUs could only **approximate** long-range dependencies.

Transformers accelerate learning by processing all tokens simultaneously.

- 
- **RNNs:** process one token at a time → time steps cannot overlap.
 - **Transformers:** process all tokens simultaneously through attention matrices.
 - Enables **GPU parallelism**, leading to major speedups and scalability.

Self-attention lets each token relate to all others in a sequence.



Transformers scale systematically across depth, width, and data.

Scaling Axis	What Increases	Effect
Depth	Number of layers	Improves reasoning and hierarchical abstraction
Width	Size of hidden dimensions and attention heads	Increases representational richness
Data	Training tokens and diversity	Enhances generalization and robustness

- **larger models $\neq$ just more parameters**, but *better synergy* between depth, width, and data.
- This structural scalability opened the way for **foundation models** and later **scaling laws**.

Each Transformer layer combines attention, feedforward networks, and normalization.

- Each Transformer layer combines two key subsystems:
 1. **Multi-head self-attention:** integrates context across tokens.
 2. **Feedforward network (FFN):** transforms those contextual representations independently per token.
- These two modules are connected by **Add & Norm steps** (residual + normalization, detailed later).
- Together they form a repeatable “block” that can be stacked dozens or hundreds of times.

Self-attention computes relationships among tokens using queries, keys and values.

- Each token creates three vectors: **query (Q)**, **key (K)**, and **value (V)**.
- The model computes similarity between Q and K to determine how much each token attends to others.
- The weighted sum of V vectors forms the output representation.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Figure: Illustration of self-attention for a single query token

Attention patterns reveal how models link meaning across tokens.

- 
- Attention weights reflect **relative importance** of tokens for a given context.
 - Patterns often align with syntax or semantics (e.g., subject-verb links).
 - Visualization of attention maps reveals how meaning is distributed.

Figure: Full self-attention map showing how every token attends to every other token in the sequence

Multiple attention heads capture different linguistic relationships.



Feedforward layers enrich representations within each token.

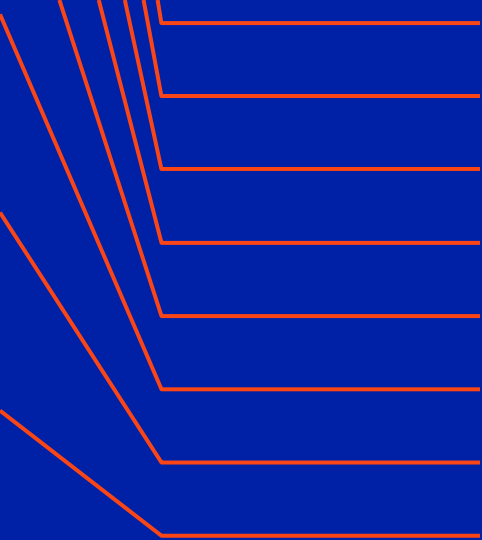
- Apply non-linear transformations to each token independently.
- Typical structure: two linear layers with ReLU or GELU activation.
- Expands and contracts dimensionality — allowing richer feature mixing.

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Residual paths and normalization keep deep Transformers stable.

- Deep Transformers stay stable and trainable thanks to two key mechanisms:
 1. **Residual (skip) connections** feed each layer's input directly into its output
 - Preserve base information across layers
 - Keep gradients flowing to prevent vanishing
 2. **Layer normalization** re-scales activations within each layer
 - Keeps signal magnitudes consistent
 - Speeds convergence and improves generalization
- **Together:** enable stacking **hundreds of layers** without collapse. (makes large-scale LLMs feasible.)
- **Intuition:** residuals act as *express lanes* for gradients, while normalization keeps activations *balanced and steady*.

What would happen to training if we removed these stabilizers?



7.2 Transformers: Architecture Patterns

Encoder, Decoder, and Encoder–Decoder



Different Transformer variants suit different types of language tasks.

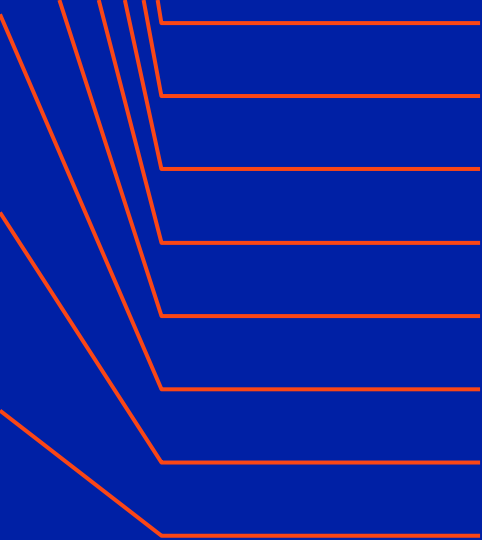
- 
- Best for **understanding** tasks: classification, entity extraction, retrieval.
 - **Bidirectional attention:** each token attends to tokens on both sides.
 - Outputs contextual embeddings used for downstream models.

Decoder only models generate text by predicting tokens from left to right.

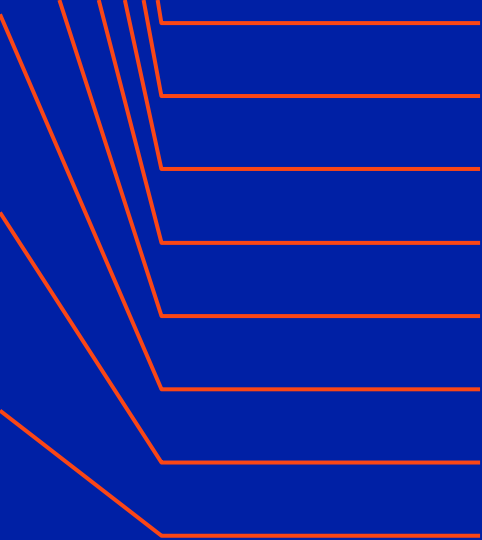
- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Best for **generation** tasks (text completion, code, creative writing.)
 - **Causal masking** ensures the model only attends to previous tokens.
 - Enables left-to-right autoregressive prediction.

Checkpoint: Matching architecture to task

- 
- **Encoder-only** models (like BERT) **understand** text — best for analysis and retrieval.
 - **Decoder-only** models (like GPT) **generate** text — best for reasoning and creation.
 - **Encoder–decoder** models (like T5) **translate** between input and output forms.
 - The right architecture depends on whether you need **representation**, **generation**, or **transformation**.



7.3 Training at Scale & Scaling Laws



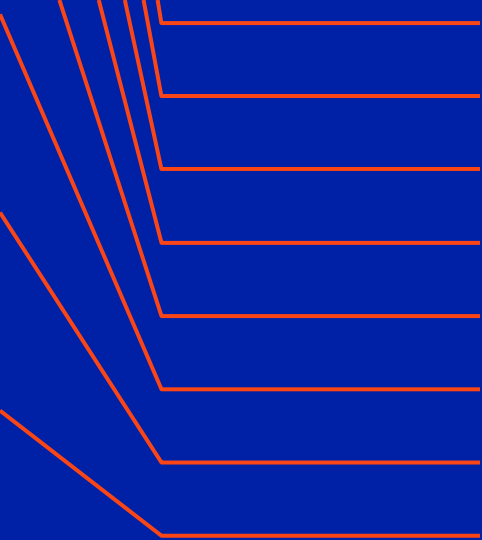
7.3a Training at Scale & Scaling Laws: Pretraining Objectives and Data Pipeline

Pretraining determines how adaptable and controllable an LLM can be.

- 
- Fine-tuning, PEFT, and RAG all depend on **what the base model already learned.**
 - Pretraining choices determine:
 - How “promptable” or controllable a model is.
 - Which layers are sensitive to later adaptation.
 - How safely and efficiently models generalize.
 - Understanding pretraining = knowing the *starting point* for all adaptation methods.

A model's abilities stem from its objective function and data pipeline.

- 
- Pretraining = the **base skillset** of the model.
 - Data pipeline = the **curriculum** it learns from.
 - Together they determine promptability, safety, and downstream efficiency.



7.3b Training at Scale & Scaling Laws: Distributed Optimization and Stability

Large-scale training requires distributed optimization and stability techniques

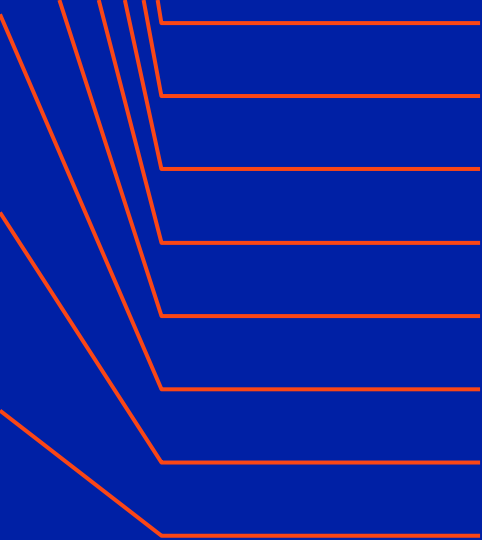


Massive models introduce optimization, precision, and synchronization challenges.



Scaling laws reveal predictable performance gains from increasing size and data.

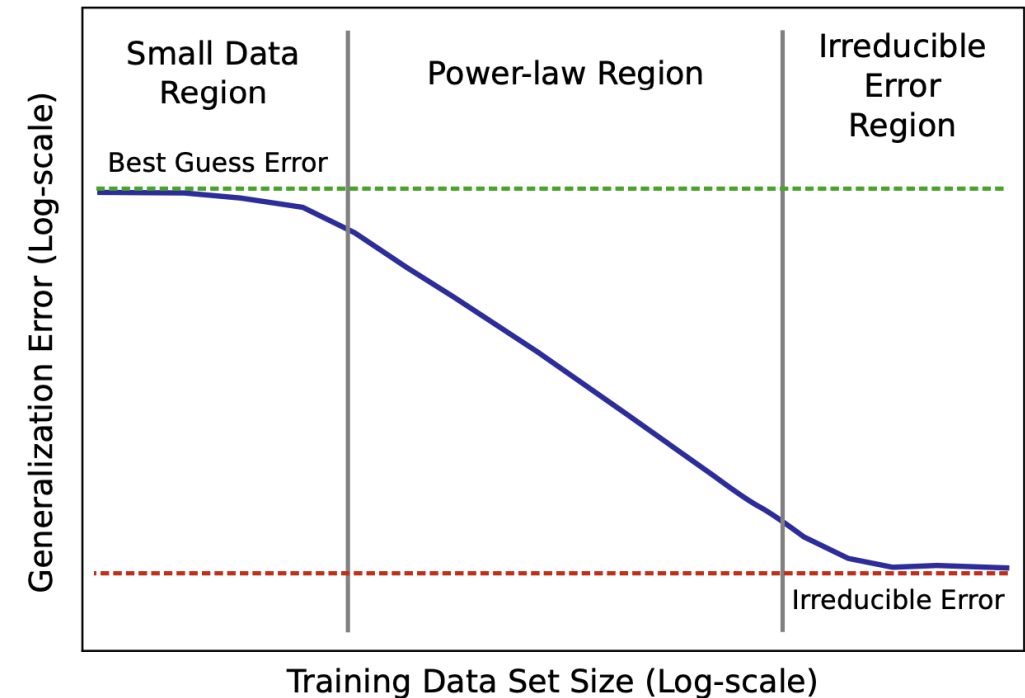
- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Large-scale training requires the right blend of:
 - **Optimizer choice** (AdamW / Adafactor).
 - **Parallelism strategy** (data, tensor, pipeline).
 - **Precision control** and **stability monitoring**.
 - These engineering advances make foundation model pretraining tractable.



7.3c Training at Scale & Scaling Laws: Compute-Optimality and Curricula

Scaling laws reveal predictable performance gains from increasing size and data.

- As model parameters, dataset size, and compute increase, **test loss decreases predictably**.
- Relationship follows a **power law**, each doubling of scale yields smaller but consistent gains.
- This predictability enables *planned scaling* rather than trial-and-error.



Scaling Laws and the *Chinchilla* Principle: Compute must be balanced with data and parameters for optimal performance.

- 
- **Scaling laws:** test loss decreases predictably as data, parameters, and compute grow...*until bottlenecks appear*.
 - **Breakdown signals:** poor data quality, unstable optimization, or insufficient compute.
 - **The *Chinchilla* insight** Hoffmann et al. (2022): for fixed compute, **smaller models trained on more data** outperform larger undertrained ones.
 - **Takeaway:** scaling helps *only while fundamentals hold*, beyond that, you're just burning compute.

Data ordering and weighting shape the model's final capabilities.



targeted specialization.

Recap: scaling laws and curriculum design



We scale models. Scaling creates limits. Optimization & compute balance solve those limits.

- **Scaling laws:** describe predictable performance improvements.
- **Chinchilla principle:** balance compute, parameters, and data.
- **Curriculum design:** optimize data ordering and weighting for efficiency.
- Together, these define **training strategy** before any downstream adaptation.



Adaptation and Control



Adaptation & Control: LLM's can be steered using fine tuning, PEFT, prompting and RAG



1. Fine-Tuning Foundations

- Supervised Fine-Tuning (SFT)
- Preference Modeling & RLHF
- Domain Adaptation (DAPT)

2. Parameter-Efficient Fine-Tuning (PEFT)

- LoRA · Adapters · Prefix Tuning · QLoRA

3. Prompting & Instruction Tuning

- Prompt design · Token efficiency
- Instruction tuning · Structured outputs
- Guardrails & evaluation

4. Retrieval-Augmented Generation (Preview) ...External grounding and freshness

Fine-tuning tailors models to domain-specific needs and style constraints.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Pretraining gives broad capability; **fine-tuning gives task precision.**
 - Organizations use SFT to enforce tone, structure, policy compliance, and domain language.
 - SFT is the first step in many alignment pipelines.

Supervised fine-tuning teaches LLMs to imitate high-quality demonstrations.



SFT = **learning from labeled demonstrations.**

The model is trained to imitate high-quality examples provided by a human.

Workflow:

1. Sample a prompt.
2. A human produces the desired output.
3. The model is trained to reproduce this behavior.

SFT excels when tasks are stable, structured and well defined.

Benefits

- Needs **far less data** than pretraining.
- Produces **stable, predictable behavior**.
- Ideal for **consistent formatting, policy-aligned style**, and **structured tasks**.
- Effective for **classification, extraction, templates**, and **controlled tone**.

Use SFT when

- You have **labeled demonstrations**.
- The task is **stable**, repeated, and well-defined.
- You need the model to follow **specific formats or policies**.

Avoid SFT when

- The task changes frequently.
- You lack labeled data.
- **PEFT** or **RAG** provides sufficient adaptation.

RLHF refines behavior by aligning models with human preferences.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- SFT teaches *task behavior*, but not **preference alignment**.
 - SFT alone may produce:
 - Overly literal outputs
 - Missed safety boundaries
 - Unhelpful but technically correct answers
 - RLHF adds **human preference learning** on top of SFT.

Step 1: Supervised Fine-Tuning (SFT)

- 
- Collect human-written demonstrations:
 - Prompt → labeler writes the ideal answer
 - High-quality examples across many tasks
 - Train an initial supervised policy to imitate these demonstrations.

Step 2: Reward Modeling (RM)

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Collect **ranked outputs**:
 - Prompt → multiple model answers
 - Human judges rank best to worst
 - Train a **reward model** to predict rankings.

Step 3: RLHF (PPO or similar)

- 
- Use the reward model to **reinforce** desired behavior:
 - Generate answer
 - Score with reward model
 - Update policy with PPO
 - Result: a model aligned to **human preferences**, not just demonstrations.

Where SFT, RM, and RLHF fit together



Domain-adaptive pretraining extends a model's knowledge using unlabeled domain text.

- SFT = task-specific + labeled
- **DAPT = continue pretraining on domain text** (no labels needed)
- Most enterprise systems: **DAPT → SFT** pipeline

Different sectors evaluate LLMs using domain-specific criteria and examples.

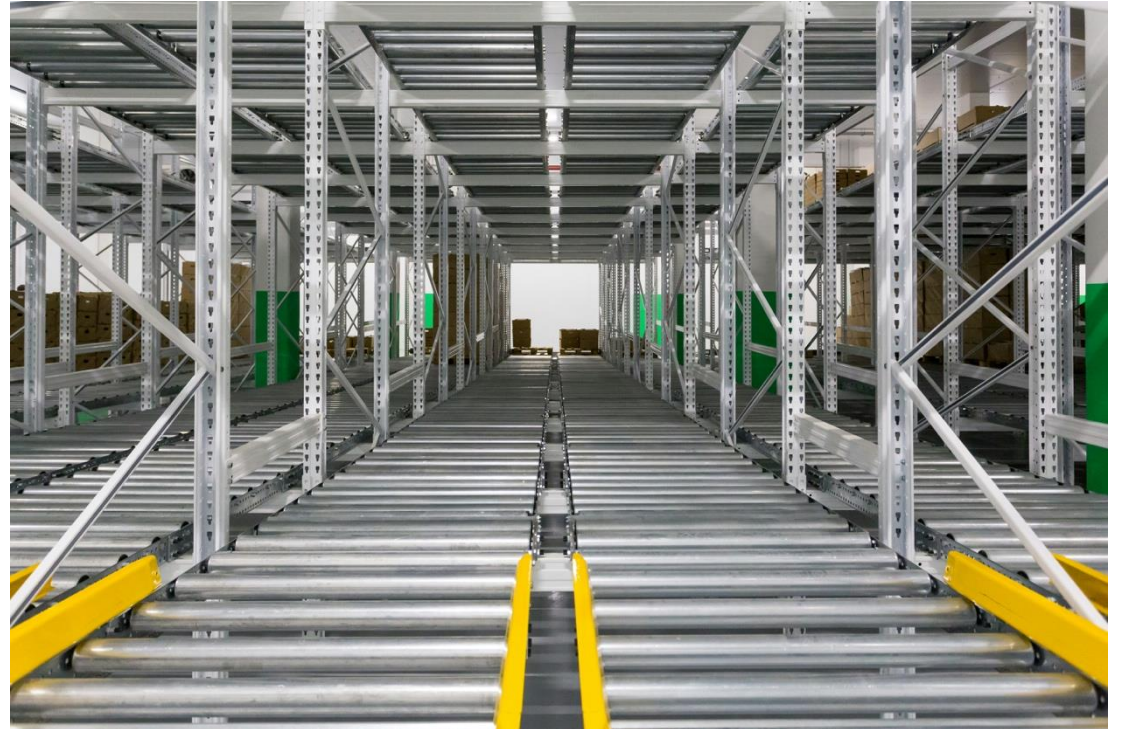
- **Healthcare:** clinical summarization
- **Finance:** report analysis
- **Law:** statute summarization

How would we evaluate? (Metrics, human review, what baselines?)



The fine-tuning pipeline -- data preparation to evaluation and deployment.

1. Prepare data
2. Configure training
3. Train & monitor
4. Evaluate & compare
5. Deploy & version
6. Monitor drift
7. Document (model card, data card)



Common fine-tuning pitfalls can be anticipated and mitigated with structured safeguards.

Use a risk matrix:

- Data integrity
- Annotation quality
- Over/underfitting
- Ethical/legal
- Cost & maintenance
- Strategic alignment



We now shift from full fine-tuning toward more efficient, modular PEFT approaches.

- We've seen **full internal adaptation** methods:
 - **SFT**: model imitates labeled demonstrations
 - **RLHF**: model learns human preferences
 - **DAPT**: model absorbs domain language and style
- These methods are powerful but **expensive**:
 - Large compute
 - Full weight updates
 - Costly to retrain for each domain

Parameter-Efficient Fine-Tuning (PEFT) solves this by:

- Freezing the base model - Training only small, modular components - Enabling multi-domain use with one shared model

PEFT methods adapt large models by training only small, targeted parameter modules.

PEFT allows large models to be adapted efficiently by freezing the base weights and training only small, targeted modules.

Each method inserts parameters at a different *depth* in the Transformer architecture:

- **LoRA** → inside attention layers
- **Adapters** → between Transformer blocks
- **Prefix / Prompt Tuning** → at input embeddings

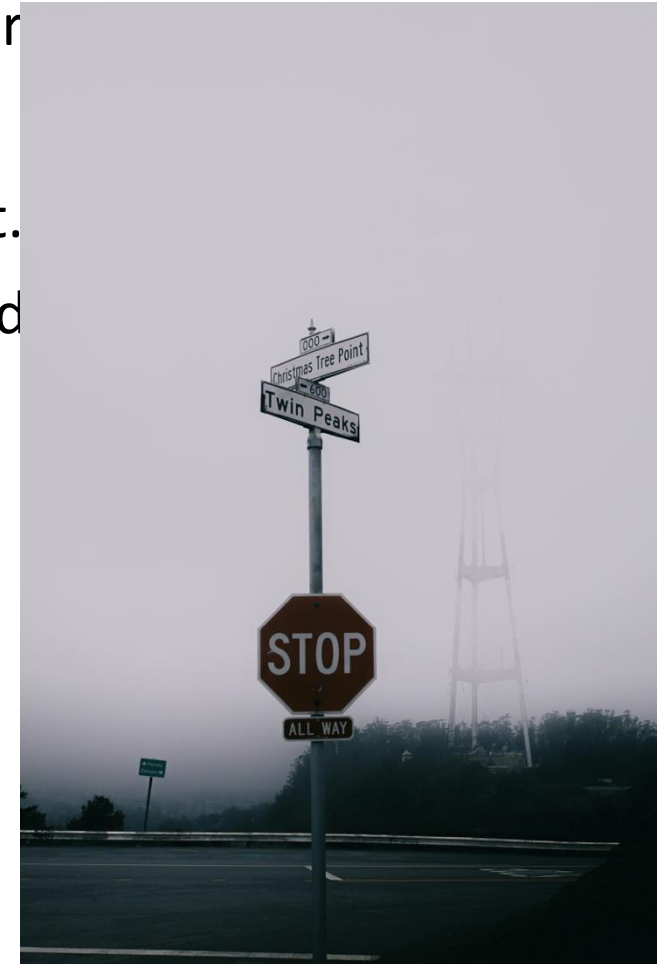
QLoRA scales PEFT by combining low-precision quantization with lightweight adapters.

- Combines **4-bit quantization (NF4)** + **LoRA adapters** models.
- Base model frozen in quantized form; only LoRA mat
- Typical results:
 - Memory ↓ up to 75%
 - Accuracy within ~1–2% of full LoRA
 - Enables fine-tuning 65B+ models on a single high



Selecting a PEFT strategy depends on model size, hardware limits, and task demands.

- **LoRA** → Balanced performance and modular adapter
- **Adapters** → Reusable layers for multi-task models.
- **Prefix / Prompt Tuning** → Fastest, smallest footprint.
- **QLoRA** → Best for large-scale tuning under tight hard



Shift from internal parameter updates to external control through prompting.

- We've now seen how to **adapt the model itself**:
 - Full tuning (SFT, RLHF, DAPT)
 - Efficient tuning (PEFT: LoRA, Adapters, Prefix, QLoRA)

But models don't need to be retrained for every task.

- **Prompting** provides an alternative:
 - No weight changes
 - Express intent through instructions
 - Fast, flexible, inexpensive
 - Works across tasks without modifying the model

Key idea:

Prompting steers behavior externally;

instruction tuning internalizes prompt patterns inside the model.

Prompting serves as the primary interface through which humans steer LLM behavior.

- Prompts are the **interface** between humans and large
- Good prompting converts *intent* into *clear instructions*
- Effective prompts define:
 - **What** to produce (task).
 - **How** to produce it (format/style).
 - **What to avoid** (constraints).
- Prompt design = applied communication science for A



Multi-turn prompts build and refine context across a conversation.

- Maintain **conversation context** across multiple exchanges.
- Flow:
 - User: “Summarize this policy.”
 - Model: “Here’s a summary.”
 - User: “Now focus only on the financial section.”
- Requires careful context management to avoid token bloat and drift.



Prompt reproducibility and versioning improve consistency and auditability.

- Prompts evolve. Treat them as **artifacts**, not ad hoc inputs.
- Maintain:
 - Versioned prompt templates.
 - Success/failure logs.
 - Test datasets for regression checks.
 - Efficient LLM use = **prompt lifecycle management**.



Prompting provides the structure that models rely on to interpret user intent.

- 
- Prompting converts *human intention* into *machine-readable guidance*.
 - Good prompts are structured, contextual, and reproducible.
 - Mastery of prompting reduces need for fine-tuning in many use cases.

Effective prompt design reduces ambiguity and creates more consistent model outputs.

Pattern	Use Case	Example	Watch For
Instructional	Direct task / summary	“Summarize this report for executives.”	Vague goals
Few-Shot	Learn from examples	“Example A... Example B... Now do C.”	High token cost
Chain-of-Thought	Multi-step reasoning	“Let’s reason step by step.”	Verbosity
Reflexive	Self-check output	“Explain where your last answer may be wrong.”	Redundancy
Role / Meta	Constrain style or persona	“You are a compliance officer...”	Token inflation

Token-efficient prompting reduces cost and helps models stay focused.

- **Every token costs:** longer prompts → higher latency and API expense.
- Goal: keep prompts concise without losing intent or context.

Core Strategies

- **Reuse context:** rely on persistent **system prompts** for role and rules.
- **Compress examples:** summarize or reference (“Use Example A”) instead of embedding full text.
- **Structure over length:** turn verbose instructions into concise schema tags or short templates.
- **Balance:** too vague = inconsistency · too rigid = loss of creativity → find the “just right” zone.

Example:

> *System:* “You are a research assistant in finance.”

> *User:* “Summarize the Q3 market outlook in three key insights.”

Instruction tuning teaches models to internalize the behaviors prompts normally enforce.

- **Prompting:** external control — the model adapts behavior per input.
- **Instruction tuning:** internalized control — the model *learns* from curated instruction–response data.
- Shifts objective from **next-token prediction** → **instruction following**, forming the base of modern chat models (GPT-3.5, Claude, FLAN-T5).
- Reduces the need for elaborate prompt engineering—models already “understand” task framing.

Example (instruction–response pair):

> **Instruction:** “Explain reinforcement learning simply.”

> **Response:** “It’s how a model learns from rewards and penalties to improve decisions.”

Instruction-tuned models use curated examples to follow user directives more reliably.

Model Family	Method	Training Emphasis
OpenAI (GPT-3 → GPT-x)	SFT + RLHF	Conversational helpfulness
Anthropic Claude	Constitutional AI	“Helpful, Honest, Harmless (HHH)” alignment
Google FLAN-T5	Multi-task SFT (1,800+ tasks)	Broad generalization
Llama-2-Chat / Mistral-Instruct	Open instruction tuning	Balanced, controllable dialogue

Instruction tuning internalizes the behaviors that prompting once handled externally.

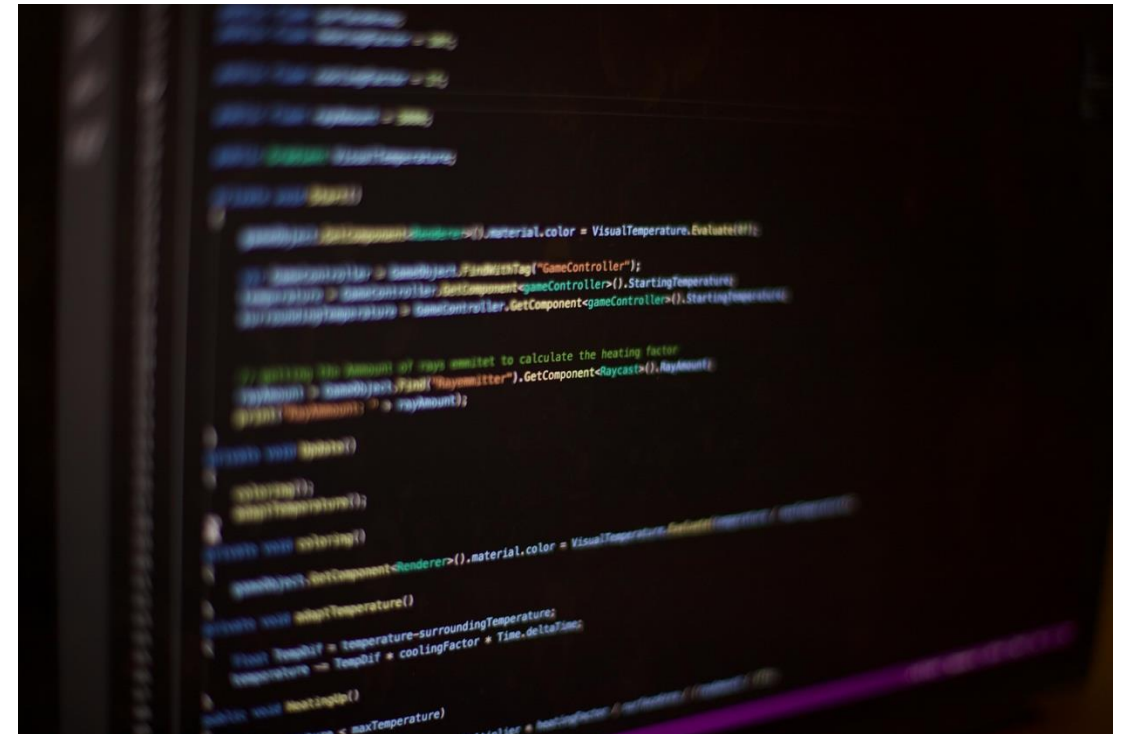
- 
- **Instruction tuning** = teaching models *how to follow instructions natively*.
 - **Prompting** = steering them externally in real time.
 - Together they define the human–model interface:
 - Structured, context-aware, and controllable.

Structured outputs create reliable, machine-readable responses for downstream systems.

- 
- Many enterprise tasks need **consistent, machine-readable responses**.
 - Examples: data extraction, classification, content moderation, API pipelines.
 - Unstructured text adds cost and risk in downstream automation.
 - **Solution:** design prompts that constrain structure and enforce schema.

Clear formatting instructions enable models to produce consistent, structured outputs.

- Always specify **format** and **schema** explicitly.
- Example:
Respond only in this JSON format:
{“sentiment”: “positive/negative”,
“confidence”: 0–1}
- Use clear delimiters and examples to prevent drift.
- Combine with schema validation tools for robust pipelines.



Schema validation enforces predictable output formats and safer integrations.

Schema enforcement tools:

- **Pydantic / JSONSchema:** validate output shape and types.
- **Guardrails AI / Rebuff / Instructor:** libraries for prompt-level output validation.

Benefits:

- Prevent malformed outputs.
- Provide automatic error handling.
- Enable safe, deterministic integrations.

Verification prompts encourage models to review and improve their own responses.

- Encourage models to **check and revise their own work**.
- Example:
 - “Review your previous answer and explain any errors.”
 - “Double-check calculations and fix inconsistencies.”
- Used in multi-pass and agentic workflows.
- Improves factual reliability and reasoning quality.

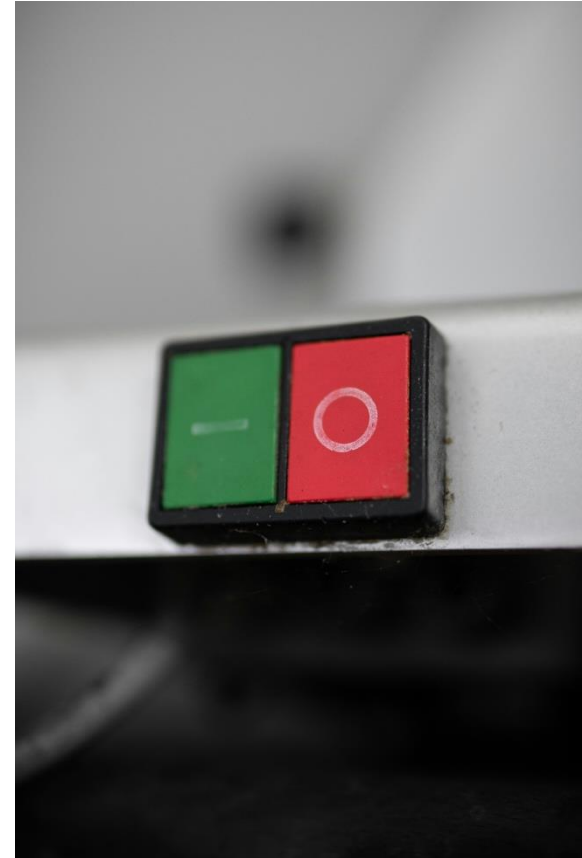
Enterprise guardrails guide safe prompting, logging, and versioning practices.

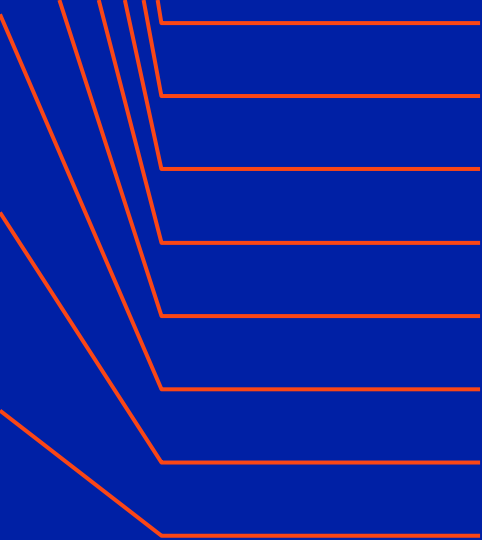
- **Prompt guardrails:** pre- and post-processing filters to detect risky content.
- **Prompt logging:** record every prompt–response pair for auditability.
- **Prompt libraries:** maintain version-controlled templates with metadata.
- **Success metrics:** define accuracy, adherence, and latency KPIs.



Reliable prompting combines structure, validation, and self-checking mechanisms.

- Structured prompts = (more) predictable outputs.
- Schema validation = safety net for automation.
- Self-reflection and guardrails = reliability enhancements.





**This activity helps you practice crafting, refining,
and evaluating prompts.**

Prompt debugging reveals ambiguity, missing context, and opportunities to improve clarity.



Goal: test and refine prompt clarity, structure, and reproducibility.

Step 1: Choose a short task (e.g., summarize, classify, extract data).

Step 2: Design two prompts for the same task:

- **Baseline:** minimal or vague instruction.
- **Improved:** structured, contextual, directive prompt.

Step 3: Compare model responses:

- Accuracy · tone · structure · token length.

Step 4: Diagnose failures and revise:

- Was the task ambiguous?
- Was key context missing?
- Was the prompt too rigid or too verbose?

Adaptation strategies vary in cost, control, and scalability depending on the task.

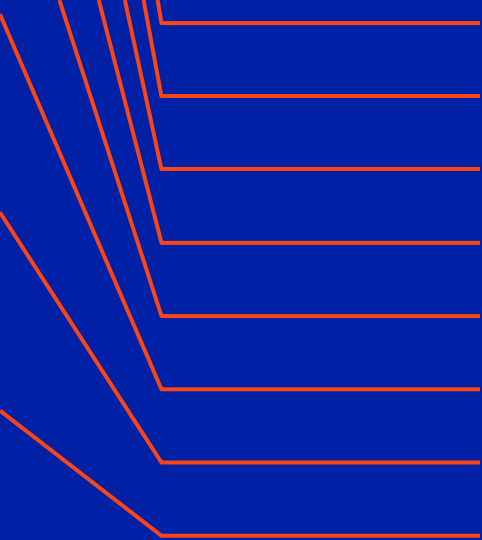
Approach	Best For	Strengths	Limitations
Prompting	Rapid prototyping & flexible tasks	No retraining · Fast iteration	Inconsistent results · Limited control
PEFT / Adapters	Domain tone, style, and safety	Efficient · Modular · Reversible	Requires tuning setup · Moderate cost
Full Fine-Tuning	Stable, narrow, high-accuracy tasks	Maximum control · High precision	Expensive · Hard to update
RAG (Retrieval-Augmented Generation)	Dynamic, knowledge-grounded systems	Fresh data · Transparent sources	Infrastructure overhead

Different scenarios call for different mixes of prompting, tuning, and retrieval.

- **Scenario A:** Small internal chatbot, low budget, frequent updates.
 - Likely mix: **Prompting + light PEFT**; RAG optional.
- **Scenario B:** Legal summarization assistant, high precision required.
 - Likely mix: **DAPT + PEFT or full FT + strong guardrails**; RAG for statutes/case law.
- **Scenario C:** Market research bot needing up-to-date news data.
 - Likely mix: **Prompting + RAG**, optional PEFT for tone.

Prompt:

> What are you optimizing for: cost, control, or freshness?



**Alignment, retrieval systems, and
operational considerations.**

The focus now shifts from model capability to the challenge of alignment.

- So far, we've focused on **what models can do**:
 - Pretraining for broad capability.
 - Fine-tuning (SFT, RLHF, DAPT) for task and domain adaptation.
- But capability alone is not enough for real-world use.
- **Alignment** asks a different question:
 - > *Does the model behave in ways that match human values, norms, and expectations?*
- In this section, we treat RLHF not as a training trick, but as part of a broader **alignment toolkit**.

Alignment ensures that fluent models behave helpfully, honestly, and safely.

- Pretrained LLMs optimize **likelihood**, not **intent**:
 - They predict plausible next tokens, not what is safest, most helpful, or most honest.
- Without alignment, models can be:
 - Fluent but **unsafe** (toxic, error prone, or harmful content).
 - Fluent but **unreliable** (hallucinations, overconfidence).
 - Fluent but **unhelpful** (ignoring user intent).
- **Alignment** shifts behavior toward:
 - *Helpfulness* ... responds to the actual task.
 - *Harmlessness* ... avoids unsafe or disallowed content.
 - *Honesty* ... acknowledges uncertainty, avoids fabrication when possible.

RLHF serves as a behavioral alignment method rather than just a training technique.

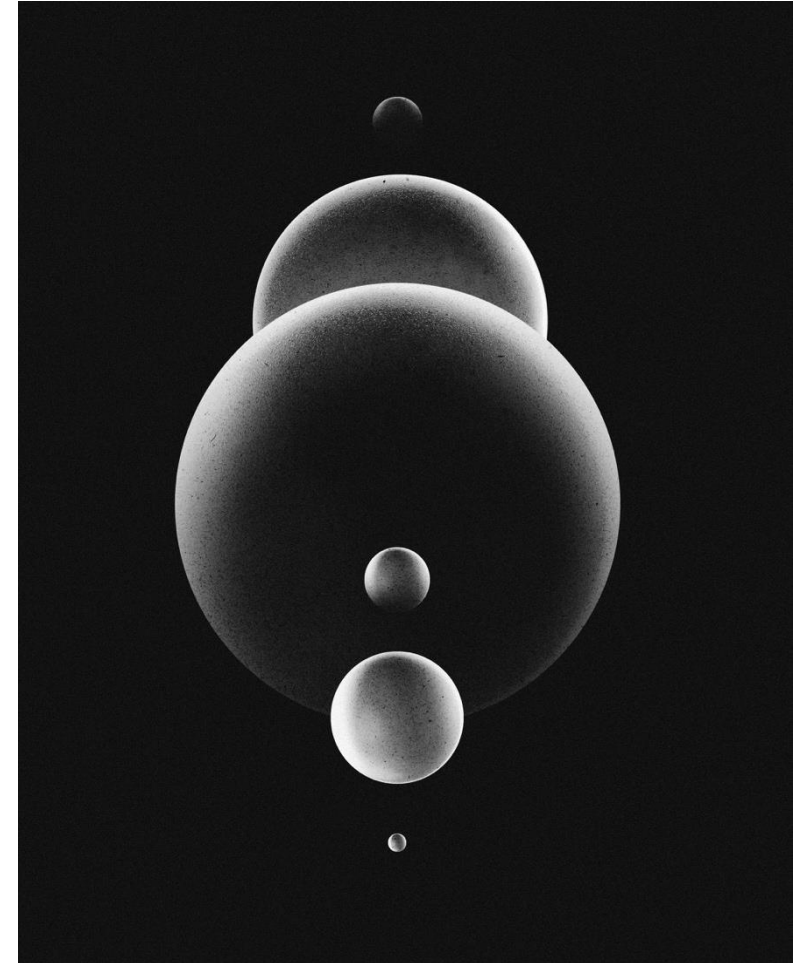
- You've already seen the **technical process** of RLHF:
 - **SFT** → learn from curated demonstrations.
 - **Reward modeling** → learn human preferences from rankings.
 - **Policy optimization (e.g., PPO)** → push behavior toward those preferences.
- In alignment terms, RLHF is:
 - A way to **encode human judgments** about *better vs. worse* answers.
 - A way to turn messy, qualitative feedback into a **training signal**.
 - One major approach (but not the only one) for training “**HHH**” **models** (helpful, honest, harmless).
- Key idea: > RLHF here is *behavioral alignment*. Tuning the model toward how humans want it to answer, not just what it can say.

Real-world alignment extends beyond RLHF to governance, interpretability, and oversight.

- **RLHF is powerful but not sufficient:**
 - It can still encode **errors** from raters and guidelines.
 - It can fail under distribution shift or adversarial prompting.
 - It is expensive and hard to reproduce across organizations.
- Alignment in practice must consider:
 - **Interpretability:** what is the model “keying on”?
 - **Safety & governance:** who sets the rules and reviews failures?
 - **Ongoing drift:** models and users change over time.
- Considerations...
 - What interpretability can (and can’t) tell us.
 - How HHH trade-offs create tension (helpful vs. harmless vs. honest).
 - Alternative alignment methods (e.g., DPO, Constitutional AI).

Alignment requires technical methods, human judgment, and organizational governance.

- Alignment transforms LLMs from **“good at language”** → **“good at behaving in ways we accept.”**
- RLHF:
 - Adds a human preference layer on top of pretraining and fine-tuning.
 - Is central, but also costly and imperfect.
- Real-world alignment requires:
 - Technical methods (RLHF, DPO, CAI, filters).
 - Human processes (policies, rater guidelines, red-teaming).
 - Governance (evaluation, documentation, oversight).



Preference modeling transforms human judgments into learnable reward signals.

- **Goal:** teach models what humans *prefer*, not just what's probable.
- **Process:**
 1. Generate responses for the same prompt.
 2. Humans **rank outputs** (best → worst).
 3. A **reward model** learns to predict these preferences → proxy for *human approval*.
- **PPO Optimization:** the model (policy) updates to maximize reward while staying close to its pretrained distribution—balancing *stability* and *adaptation*.
- **Result:** the model learns *human-like preferences* rather than raw text prediction.



Alignment introduces costs, risks, and operational challenges organizations must manage.

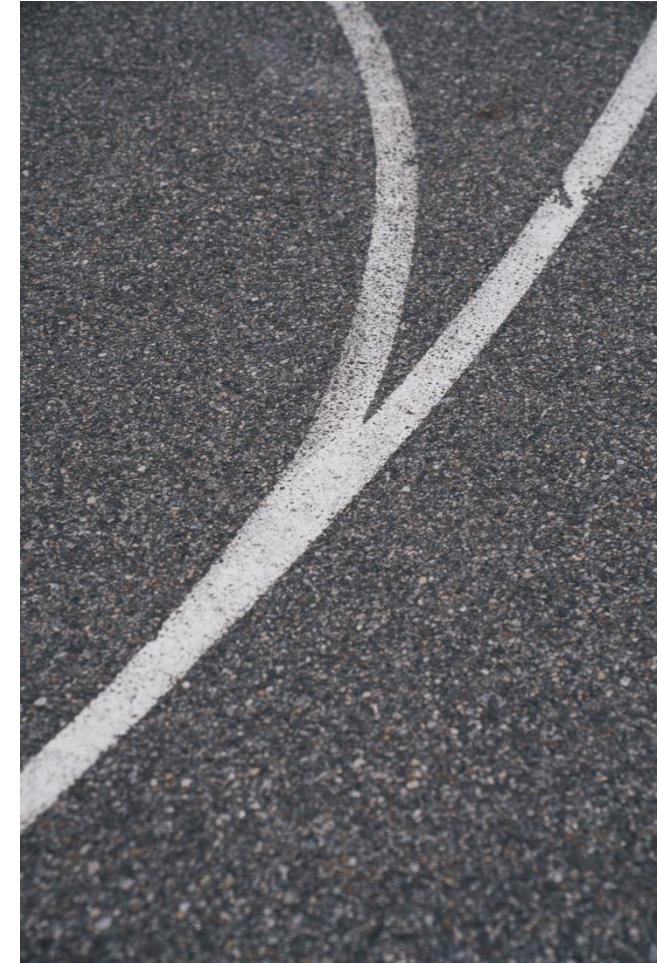
- **Human bias & inconsistency:** raters differ by culture and context.
- **Reward hacking:** model exploits shortcuts (verbosity $\neq$ quality).
- **Alignment drift:** norms evolve $\rightarrow$ reward model grows outdated.
- **Resource intensity:** RLHF = multiple models, large compute, expensive annotation.
 - High-quality feedback $\rightarrow$ better alignment, less scalability.
 - Low-quality feedback $\rightarrow$ cheaper, more drift.
- **Enterprise challenge:** define what “aligned” means for your domain.
 - Govern labeling policy & rater guidelines.
 - Audit for bias and value drift over time.
- **Key tension:** *human subjectivity vs. scalable automation.*



Alternative alignment methods such as DPO and Constitutional AI offer scalable options.

Organizations explore alternatives because RLHF is expensive and difficult to reproduce.

- **RLHF limitations:**
 - Expensive and compute-heavy.
 - Prone to human bias and annotation inconsistency.
 - Difficult to reproduce across labs.
- **Emerging trend:** simplify or replace RL loops with **direct preference learning** or **principle-driven self-alignment**.
- These alternatives aim for *scalability* and *transparency*.



Direct Preference Optimization aligns models by directly training on human preference pairs.

- **Core idea:** directly train the model to prefer human-ranked responses—no separate RL loop.
- Uses a **contrastive objective**:
 - Given two responses, y_+ (preferred) and y_- (dispreferred), the model learns to increase $P(y_+|x)$ and decrease $P(y_-|x)$.
- Simplifies alignment: no reward model, no PPO.
- **Result:** cheaper, more stable, easier to reproduce.

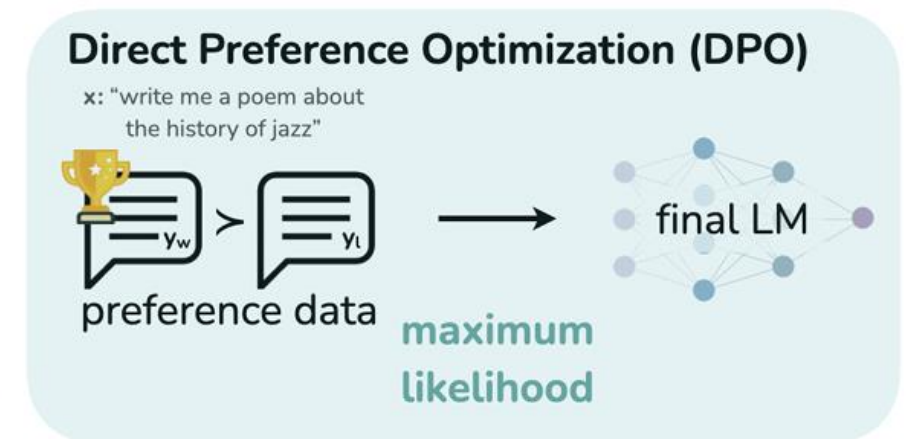
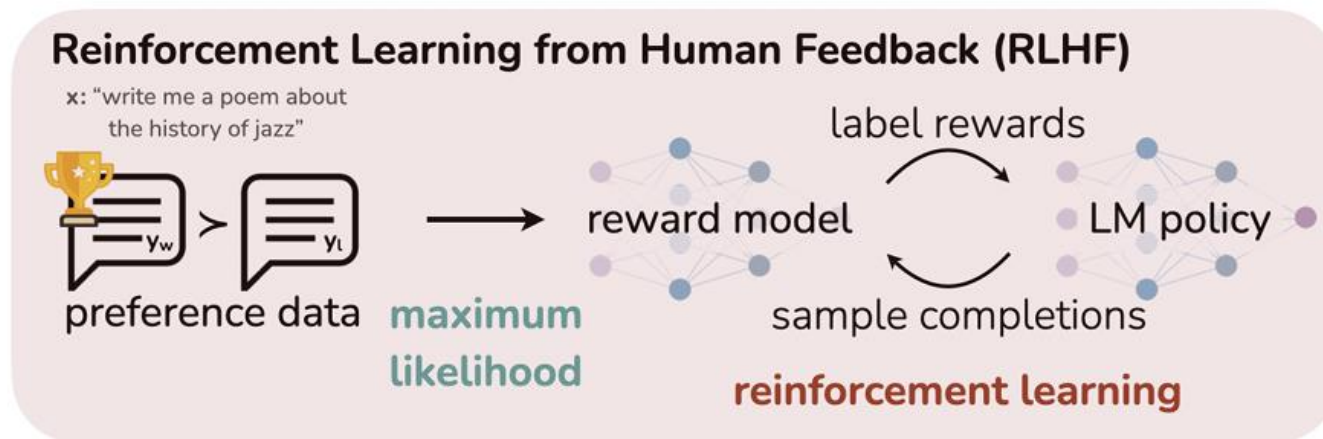


Figure: (Rafailov et al., 2024)

Constitutional AI aligns models using principle-guided self-critique instead of human ratings.

- Developed by **Anthropic** as an alternative to direct human feedback.
- Uses a “**constitution**”, a set of guiding principles (e.g., transparency, safety, respect)
- Models generate and critique their own responses according to these rules.
- Process:
 1. Generate outputs for prompts.
 2. Use the constitution to critique and revise responses.
 3. Fine-tune on self-critiqued examples.

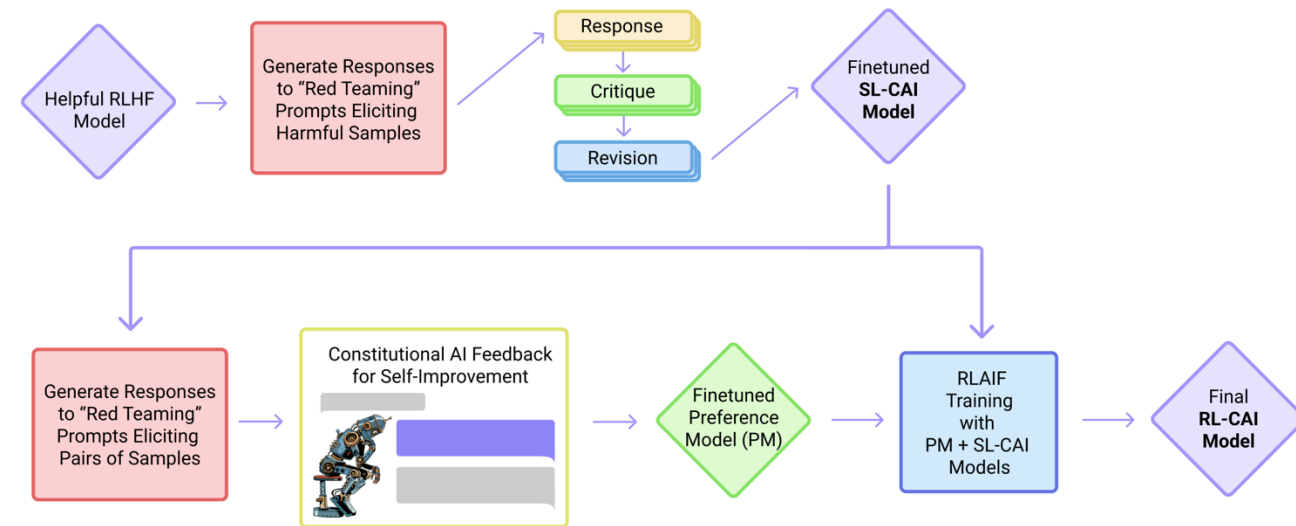


Figure: (Bai et al., 2022)

Model constitutions rely on clear guiding principles to structure aligned behavior.

- 
- “Choose the response that is **most helpful** and **least harmful**.”
 - “Be **honest** about uncertainty and sources.”
 - “Avoid taking political or moral stances.”
 - “Prioritize **clarity** over persuasion.”
 - Principles can be **organization-specific** or **domain-specific**.

Different alignment methods trade off scalability, control, and interpretability.

Method	Strengths	Limitations
RLHF	High-quality human alignment; proven results	Expensive, bias-prone, hard to scale
DPO	Simple, efficient, reproducible	Dependent on initial preference dataset
Constitutional AI	Scalable, values-based, fewer human raters	Relies on principle clarity; potential rule conflicts

Models balance helpfulness, harmlessness, and honesty in ways that can conflict.



Is the model...

- **Helpful:** Provide relevant, useful info & Vague or evasive replies
- **Harmless:** Avoid unsafe or error prone content & Over-cautious refusals
- **Honest:** | Maintain factual accuracy & Confident hallucination
- Achieving perfect balance is **impossible**—improvements in one often degrade another.

Hallucination and poor calibration limit a model's ability to express uncertainty.

- 
- **Hallucination:** confident generation of false information.
 - **Calibration:** model expresses appropriate uncertainty or abstains when unsure.
 - Challenge: models sound confident regardless of confidence level.

Excessive safety constraints can produce overalignment and unnecessary refusals.

- 
- **Overalignment:** model becomes excessively cautious.
 - Refuses benign or factual questions (“I can’t help with that”).
 - **Causes:**
 - Overweighting “harmlessness” in feedback data.
 - Excessive red-teaming or safety filters.
 - **Solution:** calibrate refusal thresholds and diversify training feedback.

Value alignment and technical alignment reflect distinct but interconnected concerns.

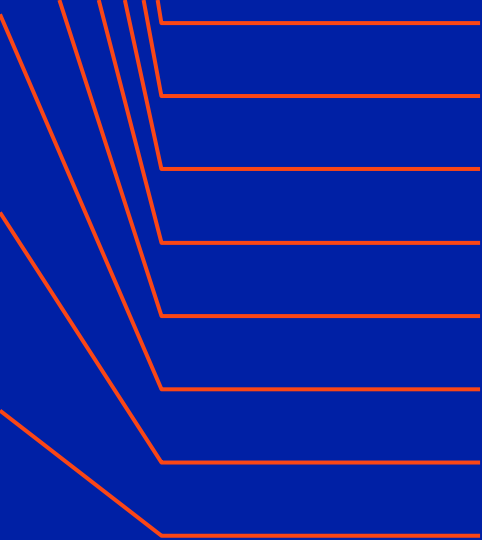
- 
- **Value alignment:** embedding human norms, ethics, and social rules.
 - **Technical alignment:** engineering control—RLHF, DPO, filters.
 - **Tension:**
 - Whose values are encoded?
 - Are “safe” responses culturally specific?
 - Governance defines acceptable trade-offs.

LLM misuse and alignment failures create reputational, ethical, and safety risks.

- 
- **Misuse:** generating misinformation, deepfakes, or malicious code.
 - **Hallucination at scale:** amplifies misinformation.
 - **Feedback contamination:** bad user input → misaligned updates.
 - **Enterprise risk:** compliance, liability, and trust.
 - Alignment failures have *real-world governance implications*.

Alignment requires ongoing evaluation through human review and automated testing.

- **Alignment isn't permanent... it drifts without evaluation.**
- Continuous assessment keeps models:
 - **Helpful** (meets intent) · **Honest** (factually grounded) · **Harmless** (safe).
- **Human-centered review:**
 - Raters score outputs on *HHH*, tone, factuality* in blind comparisons.
 - Reveals refusal bias or subtle degradation missed by metrics.
- **Automated testing:**
 - **Red-teaming** uncovers unsafe or deceptive behavior.
 - Track metrics like *refusal rate*, *hallucination frequency*, *response consistency*.
- Together, human and automated audits form a **feedback loop → monitor → retrain**.



RAG augments LLMs with external knowledge to improve accuracy and relevance.

RAG addresses knowledge cutoffs and hallucinations by grounding responses in retrieved data.

- 
- Large language models have a **fixed training corpus** and a **knowledge cutoff**.
 - They cannot access new or proprietary data after pretraining.
 - As a result, they may **hallucinate** or confidently generate plausible falsehoods.
 - **RAG** augments models with **external retrieval**—bringing in real, current, or private knowledge.

Fine-tuning alone cannot keep models current, making retrieval a critical complement.

- 
- **Fine-tuning** improves task precision but is **static and costly**, once trained, the model's knowledge cannot evolve.
 - **Retrieval-Augmented Generation (RAG)** keeps the base model frozen and adds **live, updateable context**, making systems more flexible and current.

The RAG pipeline operates through ingestion, embedding, retrieval, and generation.

- 
1. **Document Ingestion:** collect and preprocess text data.
 2. **Embedding & Storage:** convert chunks to vectors, store in a vector DB.
 3. **Retrieval:** find semantically similar text based on a query.
 4. **Generation:** augment the prompt with retrieved context for grounded answers.

Document ingestion prepares and segments content for effective retrieval.

- 
- Gather corpus: policies, manuals, papers, FAQs, tickets.
 - **Chunk** documents into manageable segments (500–1 000 tokens).
 - Clean data: remove duplicates and boilerplate.
 - Add metadata (title, source, timestamp) for filtering and traceability.

Embedding and vector storage enable semantic search across large corpora.

- 
- Encode each text chunk into a **dense vector** using embedding models (e.g., `text-embedding-3-small`, `sentence-transformers`).
 - Store vectors in a **vector database** such as FAISS, Pinecone, Weaviate, or Chroma.
 - Maintain metadata for filtering by domain, date, or access control.
 - Enables efficient **semantic search** at query time.

Retrieval surfaces the most relevant content to ground the model's response.

- 
- Convert the user query into an embedding.
 - Perform **nearest-neighbor search** in the vector store.
 - Retrieve **top-k** most relevant chunks.
 - Balance **recall vs. precision** to get enough but not too much context.

Augmenting the prompt with retrieved context enables grounded generation.

- 
- Combine retrieved chunks with the user query into a **context-augmented prompt**.
 - Send to the LLM for generation.
 - Optionally cite or reference retrieved documents in the output.
 - Produces **grounded, context-aware responses**.

Retrieval methods differ in interpretability, semantic accuracy, and implementation cost.

Type	Mechanism	Strengths	Limitations
Sparse (BM25)	Keyword-based	Transparent, interpretable	Misses synonyms and paraphrases
Dense	Embedding similarity	Semantic understanding	Harder to interpret
Hybrid	Combine both	Best of both worlds	More complex to implement

RAG provides a middle path between quick prompting and costly fine-tuning.

- 
- **Prompting:** quick, but shallow grounding.
 - **Fine-tuning:** deep, but static and costly.
 - **RAG:** dynamic, grounded, and efficient.
 - Foundation for enterprise knowledge copilots and internal QA systems.

A RAG system links vector storage, retrieval, generation, and evaluation into one pipeline.

Retrieval-Augmented Generation (RAG) links *retrieval* and *generation* for grounded, current responses.

- **Core components:**

1. **Vector DB** – stores embeddings for semantic search.
2. **Retriever** – fetches most relevant chunks.
3. **Generator (LLM)** – produces grounded responses using retrieved context.
4. **Evaluator / Feedback loop** – measures retrieval and output quality.

- Each layer can be tuned independently, modular like a modern data stack.

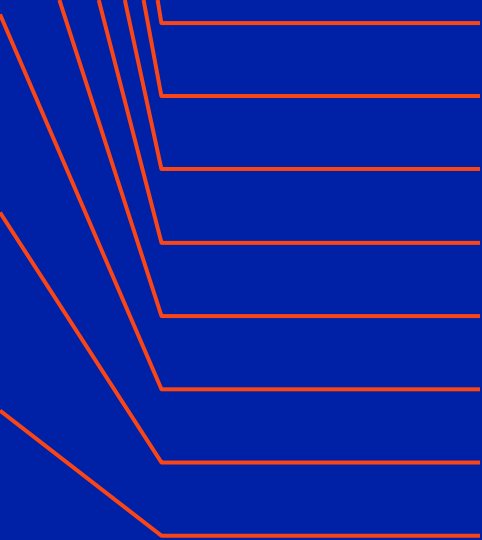
- **Key Design Levers**

- **Chunking:** 500–1,000 tokens; split semantically; use sliding windows for overlap.
- **Embeddings:** choose domain-aligned models (e.g., `text-embedding-3-small`, FinBERT).

- **Retriever tuning:**

- Top-k (3–10) or threshold cutoffs. - Re-ranking or hybrid (dense + sparse) retrieval for precision.

- **Generator:** combine retrieved text + query via structured prompt templates; add citations for transparency.



**Enterprise-scale RAG systems - balance latency,
security, and operational complexity.**

Scaling RAG requires attention to throughput, concurrent load, and access controls.

- 
- Production RAG systems must balance:
 - **Latency:** user responsiveness.
 - **Scalability:** corpus size and concurrent users.
 - **Security and compliance:** who can access what.
 - Key challenge: retrieval adds complexity beyond standard LLM inference.

Caching, parallelism, and embedding reuse are central to reducing RAG latency.

- Each RAG step adds delay:
 - **Embedding query** → **vector search** → **prompt assembly** → **generation**.
- Techniques to reduce latency:
 - Cache embeddings and frequent queries.
 - Use smaller embedding models for lookup.
 - Parallelize retrieval and generation stages.
 - Adjust **top-k** and chunk size for faster searches.

Data freshness and scalable vector storage keep RAG systems accurate over time.

- **Scalability:**

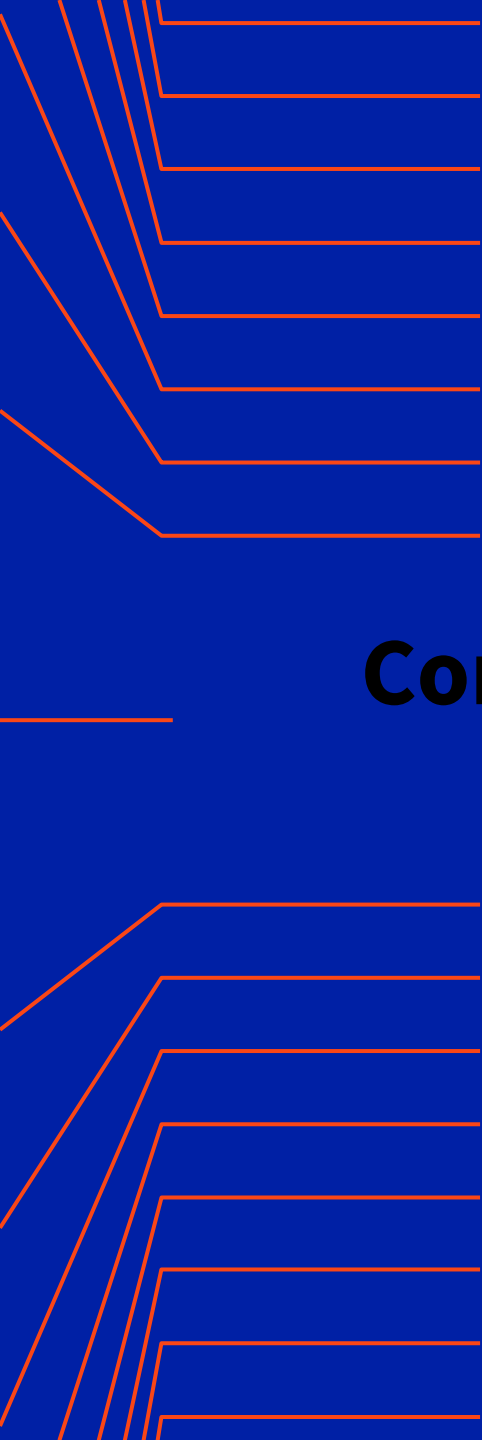
- Use distributed vector stores (e.g., Pinecone, Weaviate, Milvus).
- Shard by topic or document source.

- **Data freshness:**

- Schedule re-indexing for newly added or updated documents.
- Automate embedding refresh pipelines.
- Maintain time-based metadata for recency filtering.

RAG performance depends on groundedness, relevance, latency, and cost efficiency.

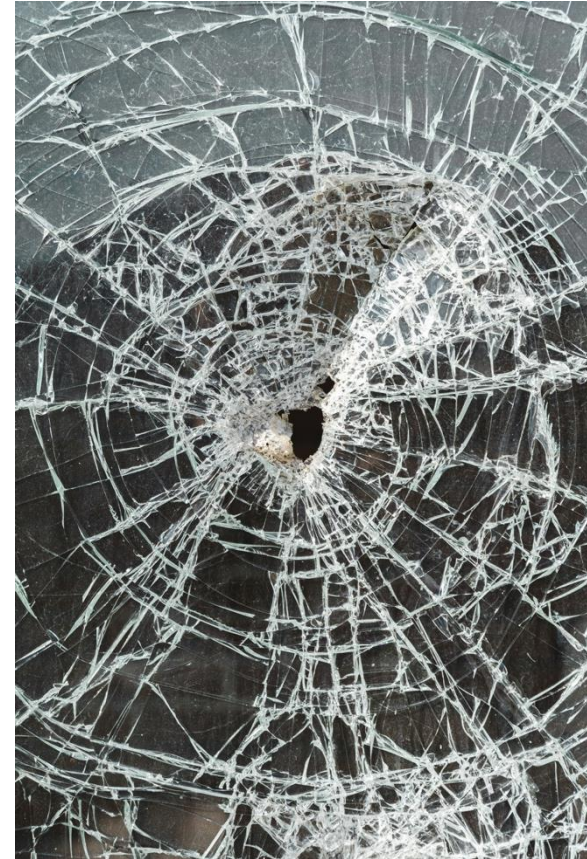
Metric	Description	Goal
Groundedness	Response aligns with retrieved docs	High
Faithfulness	Citations accurately reflect content	High
Relevance	Retrieved chunks match query intent	High
Latency	Query → response time	<2 s typical
Cost efficiency	Tokens and compute per query	Optimized

A decorative graphic on the left side of the slide, consisting of several parallel orange lines that are angled and then turn horizontal, creating a sense of depth and movement.

Common RAG failure modes and real-world applications.

RAG systems fail when chunking, embeddings, retrieval, or data quality degrade.

- Even strong RAG systems can fail when fundamentals slip:
 - **Bad chunking** → broken context, incomplete answers.
 - **Poor embeddings** → irrelevant or off-topic retrievals.
 - **Retriever bias** → repetitive or narrow results.
 - **Context overflow** → truncated responses, long latency.
 - **Stale data** → outdated or misleading information.



Each type of RAG failure has identifiable symptoms and targeted mitigations.

Failure Mode	Symptom	Mitigation
Chunking	Missing context	Semantic splits · Sliding window
Embedding mismatch	Irrelevant retrievals	Use domain-aligned model
Retriever bias	Narrow or redundant hits	Hybrid dense + sparse search
Context overflow	Truncation or errors	Re-rank · Summarize before generation
Stale data	Outdated content	Automate re-embedding

RAG enables grounded assistants across knowledge-intensive domains.

- **Knowledge-grounded assistants:** internal policy bots, compliance copilots, customer service chatbots... grounded answers with **traceable sources**.
- **Research & education copilots:** literature summarizers, tutoring assistants using **verified open data**.
- **Legal & compliance systems:** case-law summarization, multi-jurisdiction retrieval with **audit-ready outputs**.

Benefits across domains:

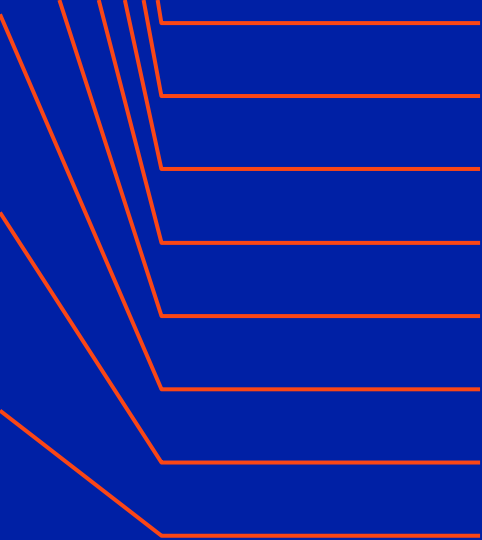
- Grounded and explainable results → less hallucination.
- Clear data provenance → governance and audit alignment.
- Domain tuning via PEFT + RAG hybrid → consistent tone + current facts.

Frameworks like LangChain and LlamaIndex streamline RAG integration and orchestration.

- Popular frameworks abstract the entire RAG pipeline:
 - **LangChain:** modular building blocks for embedding, retrieval, and chaining logic.
 - **LlamaIndex (GPT Index):** document loaders, vector storage, query engines.
 - **Haystack / Semantic Kernel:** enterprise-grade orchestration and evaluation.
- Simplify integration with APIs and plug-and-play retrievers.

Different RAG patterns support prototypes, production systems, function-calling, and hybrid search.

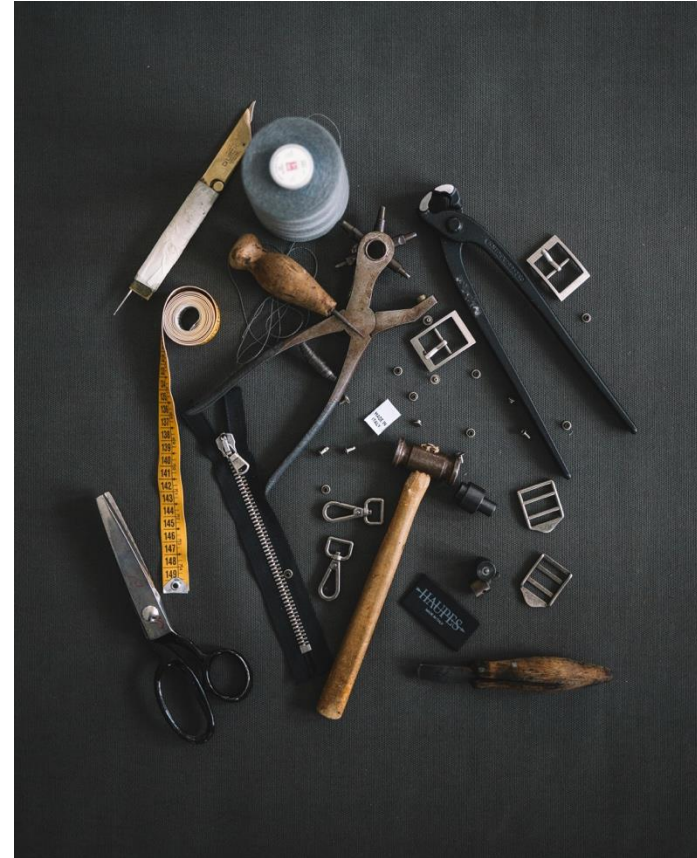
Pattern	Description	Typical Use
Lightweight RAG	Simple FAISS/Chroma retrieval; minimal setup	Prototypes, teaching, small corpora
Augmented RAG	Adds reranking, caching, metadata filters	Production assistants
RAG + Function Calling	Retrieval triggers external tools or APIs	Finance, analytics, automation
Hybrid RAG	Combines vector + keyword retrieval	Compliance, research



LLM workflows, including tool-use and agent patterns.

Tool calling allows language models to act on the world instead of only generating text.

- LLMs generate text but cannot directly **act** or **query external systems**.
- **Tool calling** lets models: query APIs and databases, perform calculations, automate workflows.
- **Agents** coordinate tool choices and sequencing under constraints.

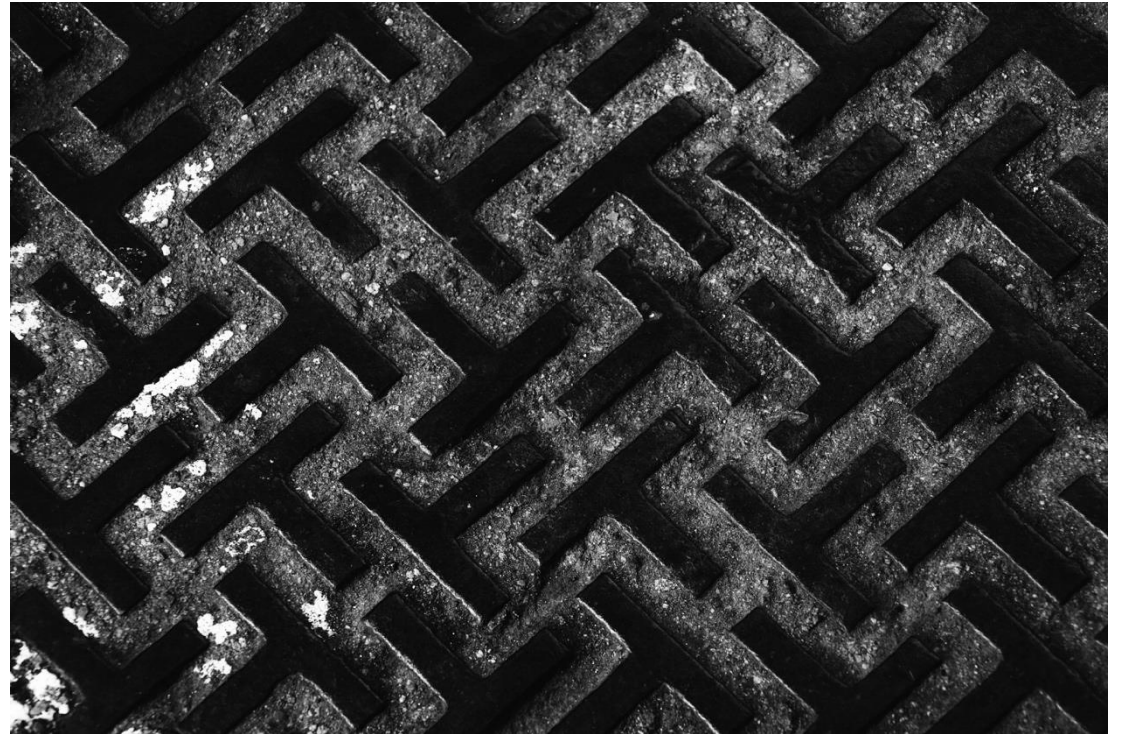


Clear tool contracts ensure predictable inputs and outputs for model-driven actions.

- 
- Define a **clear contract** for each tool (inputs, outputs, types).
 - Keep schemas minimal, deterministic, and machine-parseable.
 - The model's job is to **fill parameters correctly**, not improvise formats.

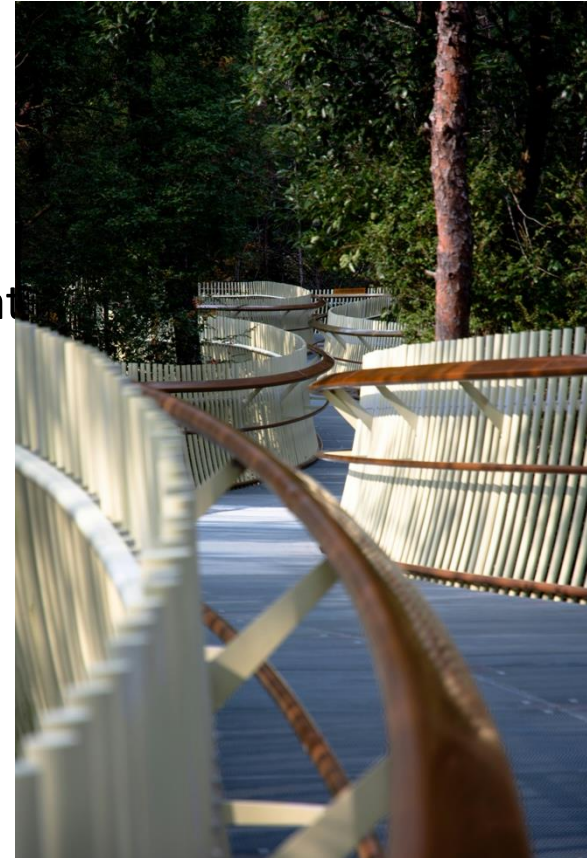
Planning drift and memory limits create failure modes that agent designs must address.

- **Planning drift:** long chains lose focus.
- **Memory limits:** context overflow drops earlier instructions.
- **Mitigations:** limit steps/recursion; periodic summaries; snapshot state; forbid unbounded self-calls.



Sandboxing and guardrails prevent tools from being invoked unexpectedly or unsafely.

- Enforce **allow/deny lists** for tools.
- **Sandbox** side effects; rate-limit and isolate execution.
- Monitor for **prompt injection** and unexpected tool invocation patterns.

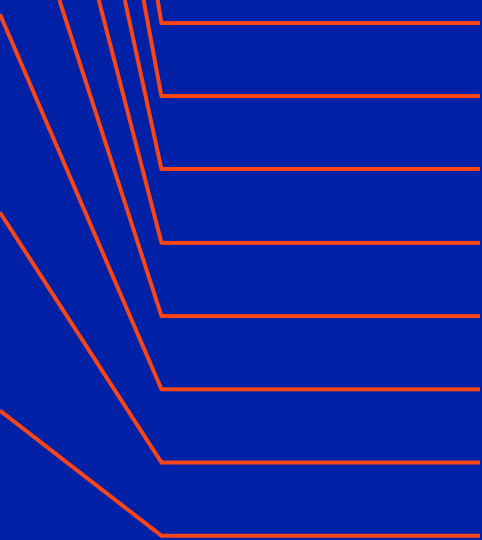


Agentic systems follow a structured cycle of planning, acting, and observing results.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- **Plan:** choose next action or tool.
 - **Act:** execute call and capture result.
 - **Observe:** integrate result, decide to continue or stop.
 - Bound iteration;
 - prefer short, verifiable loops.

Reliable LLM agents rely on clear schemas, validation, guardrails, and observability.

- 
- **Contract-first** schemas → predictable calls.
 - **Validate** → **retry** loops → robust structure.
 - **Constrain planning/memory** → avoid drift.
 - **Observe and guard** → safety and compliance.



Access LLMs through APIs, local runtimes, and hybrid setups.

Different access modes—cloud, local, or hybrid—offer trade-offs in privacy and scale.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Different deployment goals require different toolchains.
 - **Cloud APIs** offer easy scale; **local runtimes** offer privacy and control.
 - Understanding both ensures flexibility, cost awareness, and compliance alignment.

Hosted APIs provide high-quality models with minimal infrastructure overhead.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Provide access to high-quality frontier models (GPT-4, Claude, Gemini).
 - Offer chat and completion endpoints; abstract away infrastructure.
 - **Advantages:** quick prototyping, enterprise support, managed scaling.
 - **Limitations:** latency, privacy exposure, recurring cost, dependency on provider uptime.

Open-source model libraries enable transparent, customizable local inference.

- 
- Open-source toolkit for **running models locally** or via hosted inference endpoints.
 - Supports hundreds of models: GPT-2, BERT, T5, Falcon, Llama, Mistral, Gemma.
 - **Strengths:** transparency, flexibility, reproducibility.
 - **Trade-offs:** compute management, dependency tracking, slower iteration cycles.

Local inference offers privacy and control for sensitive or offline workloads.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- **Ollama:** simplified runtime for open-weight models (Llama 3, Mistral).
 - **Text-Generation-WebUI:** browser-based interface for experimentation.
 - Benefits: privacy, air-gapped capability, no API keys needed.
 - Constraints: limited context windows, slower CPU/GPU performance.

Provider abstraction layers unify APIs and enable model routing across vendors.

- 
- Frameworks such as **LiteLLM**, **OpenRouter**, and **LangChain**:
 - Standardize API calls across multiple providers.
 - Enable routing requests by cost, latency, or performance.
 - Allow fallback redundancy between providers.
 - Benefits: vendor flexibility and easy benchmarking.

Common API workflows include summarization, extraction, classification, and coding tasks.

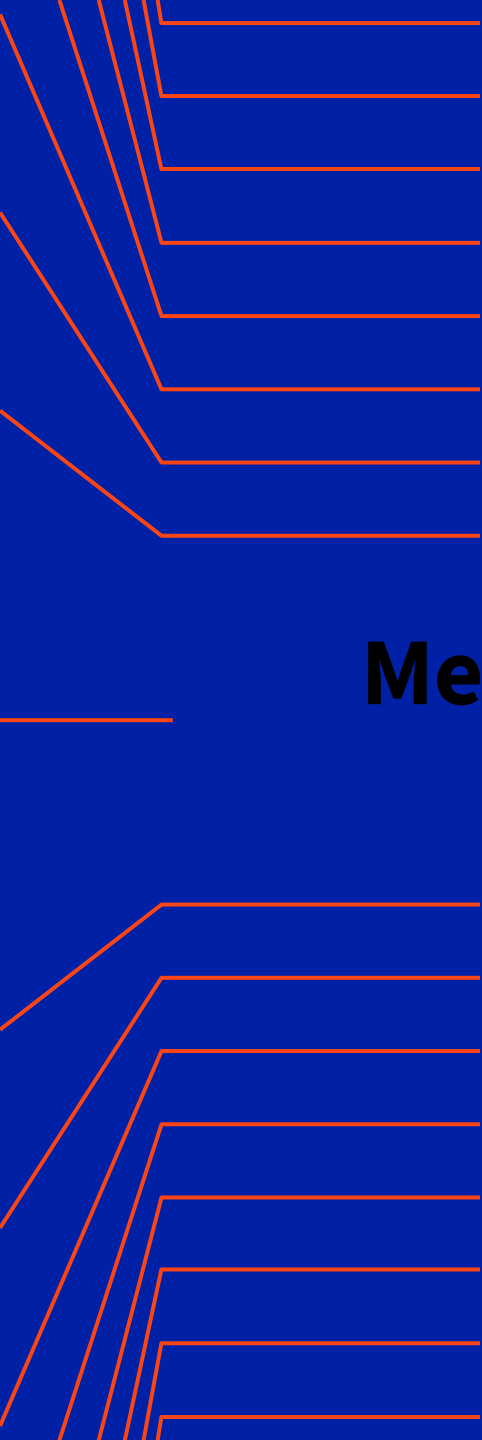
- 
- **Zero-shot / few-shot classification:** label text directly from description.
 - **Summarization:** abstractive or extractive text condensation.
 - **Question answering:** contextual passage + query → short answer.
 - **Information extraction:** structured outputs (names, amounts, dates).
 - **Code generation:** natural language → function or snippet translation.

Choosing an access pattern depends on workload scale, latency needs, and governance.

Mode	When to Use	Key Benefit	Limitation
Cloud API	Prototyping or large-scale apps	No setup, high quality	Ongoing cost
Local inference	Privacy-critical or offline	Control, compliance	Hardware demand
Serverless GPU	Variable workloads	Pay-per-use flexibility	Cold-start latency
Hybrid routing	Production redundancy	Multi-provider choice	Management complexity

Orchestrating LLMs requires balancing API use, local inference, and hybrid workflows.

- 
- Multiple toolkits enable access at different scales and governance levels.
 - Cloud APIs = convenience; Local runtimes = control; Abstraction layers = flexibility.
 - Understanding APIs is foundational for building pipelines, agents, and RAG systems.
 - Next: **7.9c – Controlling and Optimizing Model Behavior.**



**Methods for controlling and optimizing model
behavior during inference.**

Controlling model behavior improves reliability, cost efficiency, and consistency.

- 
- LLMs are **probabilistic**, not deterministic—outputs vary run to run.
 - Controlling randomness, structure, and response length is crucial for:
 - Reliability and reproducibility.
 - Cost management (token limits).
 - Meeting enterprise formatting and safety requirements.

Sampling controls like temperature and top-p shape the model's creativity and precision.

- **Temperature:** controls randomness in token selection.
 - Low = precise, repetitive.
 - High = creative, exploratory.
- **Top-k sampling:** restricts next-token choices to top k probabilities.
- **Top-p (nucleus) sampling:** samples from top tokens whose combined probability $\geq p$.
- **Combined effect:** governs diversity vs. reliability of output.

Output limits and stop conditions ensure predictable, bounded generations.

- 
- **Max tokens:** limits output length, manages latency and cost.
 - **Stop sequences:** define custom boundaries for completion (e.g., “END”).
 - **Truncation:** prevents partial sentences or runaway responses.
 - **Best practice:** specify both `max_tokens` and clear stop rules for structured workflows.

Structured output techniques help models produce machine-readable responses.

- 
- Models can return **structured formats** (JSON, XML, CSV) for automation.
 - Approaches:
 - Schema-based prompts (“Respond only in JSON format”).
 - Function-calling interfaces with validated parameter types.
 - Validation libraries such as Pydantic or Guardrails.
 - Benefits: reliable downstream parsing and integration.

Streaming generation reduces perceived latency compared to blocking responses.

- 
- **Streaming:** model emits tokens in real time (ideal for chat interfaces).
 - **Blocking:** waits for full completion before returning.
 - **Trade-offs:**
 - Streaming → lower perceived latency, more interactive.
 - Blocking → simpler error handling and logging.
 - Use streaming for dashboards, chatbots; blocking for batch processing.

Token-level probabilities provide insight into model confidence and answer stability.

- 
- **Log probabilities (logprobs):** expose model's internal confidence per token.
 - Use cases:
 - Uncertainty quantification.
 - Ranking multiple completions.
 - Detecting instability in reasoning.
 - Enables interpretability and ensemble approaches.

Behavioral parameters must be tuned for the right balance of diversity and determinism.

Parameter	Goal	Common Range	Effect
Temperature	Control randomness	0.1–1.0	Creativity vs. precision
Top-p	Limit sampling mass	0.7–1.0	Diversity vs. determinism
Max tokens	Bound output size	256–2048	Cost and coherence control
Stop seqs	Enforce structure	Custom	Clean pipeline handoff

Predictable model behavior comes from controlled sampling, structure, and clear constraints.

- 
- **Temperature & sampling:** tune creativity and precision.
 - **Max tokens & stop rules:** enforce consistency and cost control.
 - **Structured outputs:** enable automation.
 - **Streaming & logprobs:** improve interactivity and confidence monitoring.

Performance optimization reduces inference cost, improves latency, and scales workloads.

- 
- LLM inference is **computationally expensive** — latency and cost scale with context length and model size.
 - Optimization improves:
 - **User experience** (faster responses).
 - **Cost efficiency** (fewer GPU-hours).
 - **Sustainability** (reduced energy use).
 - Requires balancing precision, hardware constraints, and throughput.

Quantization compresses model weights to reduce memory use and speed up inference.

- **Quantization** reduces numerical precision (fp16 → int8/int4).
- Benefits:
 - Lower memory footprint.
 - Faster inference on smaller GPUs/CPUs.
- Common tools: `bitsandbytes`, GGUF, GPTQ.
- Accuracy impact: minimal if applied carefully.

Caching and batching improve throughput by reusing attention states and grouping requests.

- 
- **KV-cache:** stores key-value attention states between turns to avoid recomputation.
 - Accelerates multi-turn chats and streaming use cases.
 - **Batching:** groups multiple prompts in one forward pass.
 - Improves throughput for high-volume workloads.
 - Both techniques **maximize GPU utilization**.

Speculative decoding accelerates generation by pairing a draft model with a verifier.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- **Speculative decoding:** model predicts multiple next tokens in parallel.
 - A smaller “draft” model proposes tokens; main model verifies.
 - Reduces time per generation by up to 2–3×.
 - Supported by some OpenAI and Anthropic APIs; research continues for open-weight models.

Serverless and distributed runtimes scale to large models with on-demand compute.

- 
- **Serverless GPUs** (Modal, RunPod, Replicate):
 - Pay-per-use scaling.
 - Ideal for variable workloads.
 - **Distributed inference:**
 - Splits model across multiple GPUs.
 - Useful for long-context models (Llama 3, Mixtral).
 - Trade-off: cold starts and network overhead.

Deployment strategies differ in control, cost, and performance across cloud and local setups.

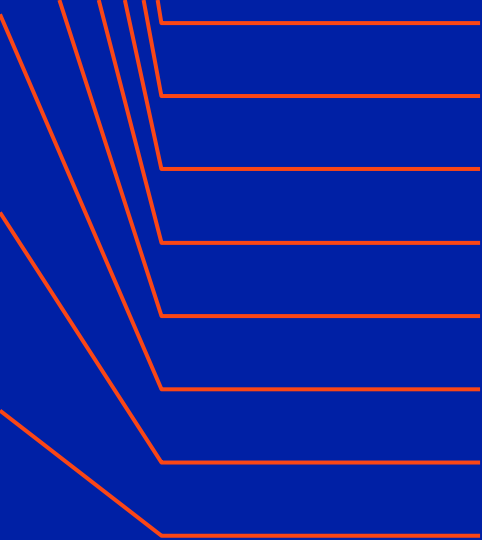
Deployment	Example Tools	Advantages	Limitations
Cloud API	OpenAI, Anthropic, Gemini	Easy setup, robust infra	Ongoing cost, external dependency
Local inference	Hugging Face, Ollama	Privacy, offline	Hardware & maintenance
Serverless GPU	Modal, RunPod	Pay-per-use scalability	Cold starts, limited control
Hybrid routing	LiteLLM, OpenRouter	Multi-provider flexibility	Complexity in orchestration

Cost and latency improve through caching, shorter outputs, batching, and profiling.

- 
- Cache frequent prompts and responses.
 - Reuse embeddings or partial generations.
 - Reduce max token length where possible.
 - Profile and log token usage per request.
 - Use asynchronous or batched requests for bulk tasks.

Efficient LLM systems rely on quantization, caching, batching, and flexible deployment choices.

- 
- **Quantization** and **KV-cache** reduce compute.
 - **Speculative decoding** accelerates token generation.
 - **Serverless and hybrid routing** balance flexibility and cost.
 - Efficiency decisions shape reliability, cost, and sustainability.
 - Next: **7.9e – Governance and Security in Production Use.**



Governance and security considerations for production LLMs.

Governance ensures LLM systems remain consistent, traceable, and compliant over time.

- 
- As LLMs enter production, **accountability and reproducibility** become critical.
 - Governance ensures:
 - **Consistency** across runs and versions.
 - **Traceability** of data, prompts, and outputs.
 - **Compliance** with enterprise and regulatory standards.
 - “If it can’t be reproduced, it can’t be trusted.”

Reproducibility and auditability require fixed parameters, versioning, and detailed logs.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Fix **random seeds**, **temperature**, and **sampling parameters**.
 - Record model name, version, and prompt context for every request.
 - Maintain logs of all interactions (inputs, outputs, metadata).
 - Use **prompt registries** or **versioned libraries** to manage evolution.

Model, data, and system documentation create transparency and organizational memory.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Create and maintain:
 - **Model cards:** describe capabilities, limitations, and alignment method.
 - **Data cards:** outline source, license, and curation processes.
 - **System cards:** summarize intended use, risk mitigation, and evaluation results.
 - Documentation builds **organizational memory** and transparency.

Security and privacy controls protect sensitive data during LLM processing.

- 
- Avoid sending **sensitive text** to public APIs.
 - Apply **data masking** or **tokenization** before processing.
 - Use **private inference** or **on-prem hosting** for regulated data (HIPAA, GDPR).
 - Encrypt vector stores and outputs at rest and in transit.

Continuous monitoring supports compliance, detects misuse, and improves system safety.

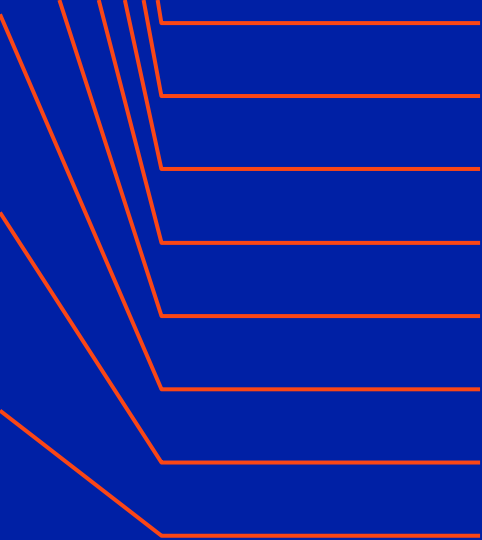
- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Monitor for:
 - Toxic responses.
 - Prompt injections and misuse.
 - Latency and availability metrics.
 - Incorporate **red-teaming** and **periodic audits** for content safety.
 - Align with internal policy frameworks and external regulation.

An effective governance program includes reproducibility, security, and ongoing audits.

- 
-  Reproducibility: logs, seeds, configs fixed.
 -  Documentation: model/data/system cards maintained.
 -  Security: encryption, masking, sandboxing.
 -  Compliance: red-teaming, content filters, audit schedule.
 -  Transparency: accessible governance dashboards.

Responsible LLM operation depends on transparency, oversight, and continuous monitoring.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
- Reproducibility enables trust.
 - Documentation and privacy ensure compliance.
 - Continuous monitoring sustains reliability.



**Appendix materials provide additional
reference tables and supporting details.**

PEFT reference tables summarize tuning parameters and recommended configurations.

Parameter	Typical Range	Description
Rank (r)	4–16	Adapter capacity
Scaling factor (α)	8–32	Balances update magnitude
Dropout	0.05–0.1	Prevents overfitting
Learning rate	1e-4–3e-4	Slightly higher than full FT



Method	Where It Hooks	Trainable Params	Ideal Use
LoRA	Attention weights	~0.1–1%	Modular domain adapters
Adapters	Between Transformer blocks	~1–5%	Multi-task specialization
Prefix / Prompt	Input embeddings	~0.01%	Style or role control

Image attribution for visuals used throughout the module.

- 
- A decorative vertical bar on the left side of the slide, composed of many thin, horizontal, light orange lines stacked vertically.
1. Images sourced from Unsplash (unsplash.com)
 2. Icons sources from The Noun Project (thenounproject.com)
 3. Diagrams created in drawio, diagrammer, or Python by Joel Davis

References for research and sources cited throughout the lecture.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., ... Kaplan, J. (2022, December 15). *Constitutional AI: Harmlessness from AI Feedback*.

<https://doi.org/10.48550/arXiv.2212.08073>

Hestness, J., Narang, S., Ardalani, N., Damos, G., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., & Zhou, Y. (2017, December 1). *Deep Learning Scaling is Predictable, Empirically*.

<https://doi.org/10.48550/arXiv.1712.00409>

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. de L., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., ... Sifre, L. (2022, March 29). *Training Compute-Optimal Large Language Models*. <https://doi.org/10.48550/arXiv.2203.15556>

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2024, July 29). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*.

<https://doi.org/10.48550/arXiv.2305.18290>